

Guide de mise a jour du serveur AirBar (upgrade depuis GitHub)

Objectif: recuperer une nouvelle version du code depuis le repository GitHub et la deployer sans perte de donnees, avec possibilite de revenir en arriere en cas de probleme.

Prerequis: la stack tourne deja via Docker Compose (voir [installation_windows.md](#)).

1. Verifier l'etat actuel avant upgrade

Depuis `C:\airbar_production\airbar_backend\airbar_backend_server`:

```
docker compose -f .\docker-compose.yaml ps
git status
git log -1 --oneline
```

Notez le commit actuel: il servira de point de retour si l'upgrade echoue.

2. Sauvegarder la base de donnees (obligatoire)

Ne jamais upgrader sans backup prealable.

Creez d'abord le dossier `backups` s'il n'existe pas encore:

```
mkdir backups -ErrorAction SilentlyContinue
```

Puis effectuez le dump et copiez-le localement:

```
docker exec airbar_postgres_prod pg_dump -U postgres -d airbar_backend -F c -f /tmp/airbar_backup.dump
```

```
docker cp airbar_postgres_prod:/tmp/airbar_backup.dump  
.\backups\airbar_backup_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').dump
```

3. Sauvegarder les fichiers locaux non versionnés

Ces fichiers ne viennent pas de Git et doivent être préservés :

- `.env`
- `config\production.yaml` (si personnalisé localement)

```
copy .env .env.backup  
copy config\production.yaml config\production.yaml.backup
```

4. Récupérer la nouvelle version du code

```
cd C:\airbar_production\airbar_backend  
git fetch origin  
git status
```

Si vous avez des modifications locales non committées et non voulues, vérifiez-les avant de continuer (`git diff`). Sinon :

```
git pull origin main
```

5. Comparer les fichiers sensibles

Après le pull, vérifiez si `docker-compose.yaml`, `Dockerfile` ou `config\production.yaml` ont changé (nouvelles variables d'environnement, nouveaux secrets requis, etc.) :

```
git log -p -3 -- airbar_backend_server/docker-compose.yaml
git log -p -3 -- airbar_backend_server/config/production.yaml
```

Si de nouvelles variables `SERVERPOD_PASSWORD_*` ont été ajoutées, complétez votre `.env`.

6. Reconstruire l'image et redémarrer

```
cd .\airbar_backend_server
docker compose -f .\docker-compose.yaml up -d --build
docker compose -f .\docker-compose.yaml ps
```

7. Appliquer les nouvelles migrations (si présentes)

Vérifiez d'abord si de nouvelles migrations sont arrivées:

```
git log -1 --oneline -- migrations\migration_registry.txt
```

Si oui, appliquez-les en mode production:

```
docker compose -f .\docker-compose.yaml run --rm --entrypoint /bin/sh airbar_server
-c "./server --mode=production --server-id=prod-001 --logging=normal --
role=monolith --apply-migrations"
docker compose -f .\docker-compose.yaml up -d airbar_server
```

8. Vérifier que tout fonctionne

```
docker compose -f .\docker-compose.yaml logs --tail 100 airbar_server
```

Attendu: pas d'erreur `relation ... does not exist`, pas de boucle de restart, le serveur annonce bien `runMode: production`.

Test rapide de l'API:

```
curl http://localhost:8080
```

9. En cas de probleme: revenir en arriere

Revenir au code precedent

```
cd C:\airbar_production\airbar_backend  
git checkout <commit-note-etape-1>  
cd .\airbar_backend_server  
docker compose -f .\docker-compose.yaml up -d --build
```

Restaurer la base si une migration a corrompu les donnees

```
docker compose -f .\docker-compose.yaml stop airbar_server  
docker cp .\backups\<fichier_backup>.dump airbar_postgres_prod:/tmp/restore.dump  
docker exec airbar_postgres_prod pg_restore -U postgres -d airbar_backend --clean -  
-if-exists /tmp/restore.dump  
docker compose -f .\docker-compose.yaml up -d airbar_server
```

10. Check-list upgrade

- ☐ Commit actuel note
- ☐ Backup base de donnees effectue
- ☐ .env sauvegarde
- ☐ git pull effectue sans conflit
- ☐ Fichiers sensibles (docker-compose.yaml, production.yaml) verifies
- ☐ Image reconstruite (--build)
- ☐ Migrations appliquees si necessaire
- ☐ Logs verifies (pas de crash)

- ☐ API testée et fonctionnelle
- ☐ Plan de rollback identifié (commit + backup) au cas où